Switch-Level Modeling in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Switch-Level Syntax **3**

3 MOS Switch Primitives **4**
3.1 NMOS Switch . . . 4
3.1.1 Example: NMOS Switch . . . 4
3.2 PMOS Switch . . . 4
3.2.1 Example: PMOS Switch . . . 4

4 NMOS and PMOS Comparison **5**

5 CMOS Switch Primitive **5**
5.1 Syntax . . . 5
5.2 Example: CMOS Switch . . . 5

6 Transmission Gate **6**
6.1 Example: Transmission Gate . . . 6

7 Bidirectional Switch Primitives **6**
7.1 The tran Primitive . . . 6
7.1.1 Syntax . . . 6
7.1.2 Example . . . 6
7.2 The tranif1 Primitive . . . 7
7.3 The tranif0 Primitive . . . 7

8 Resistive Switches **8**
8.1 Example: Resistive NMOS . . . 8
8.2 Example: Resistive Bidirectional Switch . . . 8

9 Pull-Up and Pull-Down Primitives **9**
9.1 Pull-Up Primitive . . . 9
9.2 Pull-Down Primitive . . . 9
9.3 Pull-Up and Pull-Down Comparison . . . 9

10 Example: Switch-Level AND Gate **9**
10.1 NAND Gate Using Switches . . . 10

11 Example: Switch-Level Multiplexer **10**
11.1 Verilog Code 10

12 Testbench for Switch-Level Modeling **11**
12.1 Example: NMOS Testbench 11

13 Logic Strengths and High-Impedance State **12**

14 Switch-Level Primitive Summary **12**

15 Switch-Level Modeling vs. Other Modeling **13**

16 Limitations of Switch-Level Modeling **13**

17 RTL Design Guidelines **13**

18 Summary **14**

1 Introduction

Switch-level modeling is a modeling technique in Verilog HDL used to represent digital circuits using transistor-like switches and electrical connections between signals.

It is useful for understanding how MOS transistors, pass transistors, bidirectional switches, and pull-up or pull-down devices behave in a digital simulation.

Unlike behavioral modeling, which describes what a circuit does, switch-level modeling describes the circuit using primitive devices and their connections.

Verilog provides built-in switch primitives such as:

- MOS switches: `nmos`, `pmos`
- CMOS switch: `cmos`
- Resistive switches: `rnmos`, `rpmos`, `rtran`
- Bidirectional switches: `tran`, `tranif0`, `tranif1`
- Pull devices: `pullup`, `pulldown`

Switch primitives are built into Verilog. They do not need to be defined using a separate `primitive` declaration.

2 Basic Switch-Level Syntax

A MOS switch can be instantiated using the following general syntax.

```
nmos instance_name (output_node, input_node, control);

pmos instance_name (output_node, input_node, control);
```

For example:

```
module nmos_example (y, a, en);

    input a, en;
    output y;

    wire y;

    nmos n1 (y, a, en);

endmodule
```

The MOS switch connects the first two terminals when its control input activates the device.

The exact terminal ordering depends on the primitive type. For MOS primitives, the order is drain, source, and gate.

For Verilog switch primitives, the first two terminals of a MOS primitive are represented as the two conduction terminals. Their electrical behavior is modeled by the Verilog simulator.

3 MOS Switch Primitives

3.1 NMOS Switch

An NMOS switch is controlled by a gate signal.

- When the gate is 1, the switch is on.
- When the gate is 0, the switch is off.
- When the gate is unknown or high impedance, the result depends on the switch's control interpretation and the connected signals.

3.1.1 Example: NMOS Switch

```
module nmos_switch (y, a, en);

    input a, en;
    output y;

    nmos n1 (y, a, en);

endmodule
```

When `en = 1`, the NMOS switch provides a conducting path between `a` and `y`. When `en = 0`, the switch is off.

An NMOS pass transistor generally passes a strong logic zero and a degraded logic one in physical circuits. Standard Verilog switch primitives use discrete logic strengths and do not model analog voltage degradation continuously.

3.2 PMOS Switch

A PMOS switch has complementary control behavior.

- When the gate is 0, the switch is on.
- When the gate is 1, the switch is off.

3.2.1 Example: PMOS Switch

```
module pmos_switch (y, a, en);

    input a, en;
    output y;

    pmos p1 (y, a, en);

endmodule
```

The PMOS switch conducts when its control input is zero.

A PMOS pass transistor generally passes a strong logic one and a degraded logic zero in physical circuits. Verilog switch-level simulation models discrete logic values and strengths rather than continuous transistor voltages.

4 NMOS and PMOS Comparison

Control	NMOS	PMOS
0	OFF	ON
1	ON	OFF

NMOS and PMOS switches have opposite control polarities. They can be combined to implement CMOS switching structures.

5 CMOS Switch Primitive

The `cmos` primitive combines an NMOS and a PMOS switch under complementary control.

5.1 Syntax

```
cmos instance_name (output_node, input_node,
                    ncontrol, pcontrol);
```

The CMOS switch conducts when the NMOS control is high and the PMOS control is low.

5.2 Example: CMOS Switch

```
module cmos_switch (y, a, n_en, p_en);

    input a, n_en, p_en;
    output y;

    cmos c1 (y, a, n_en, p_en);

endmodule
```

For normal complementary control signals:

- `n_en = 1` activates the NMOS path.
- `p_en = 0` activates the PMOS path.
- Both paths conduct when both conditions hold.
- Both paths are off when the NMOS control is low and the PMOS control is high.

The `cmos` primitive models a switch structure. It does not model transistor currents, capacitances, or analog voltage levels.

6 Transmission Gate

A transmission gate uses parallel NMOS and PMOS switches with complementary control signals.

It can transfer both logic zero and logic one more effectively than a single pass transistor in a conventional CMOS circuit.

6.1 Example: Transmission Gate

```
module transmission_gate (y, a, en, en_bar);

    input a, en, en_bar;
    output y;

    nmos n1 (y, a, en);
    pmos p1 (y, a, en_bar);

endmodule
```

Here, `en` controls the NMOS switch and `en_bar` controls the PMOS switch.

For normal operation:

- `en = 1` and `en_bar = 0`: the transmission gate is ON.
- `en = 0` and `en_bar = 1`: the transmission gate is OFF.

The complementary signals should be generated or supplied correctly. If they are not complementary, the switch may not behave as intended.

7 Bidirectional Switch Primitives

Bidirectional switches allow a signal to pass between two nets in either direction.

Unlike a normal MOS primitive, which has a control terminal and two conduction terminals, a basic `tran` primitive has two bidirectional terminals and no control input.

7.1 The tran Primitive

7.1.1 Syntax

```
tran instance_name (net1, net2);
```

7.1.2 Example

```
module tran_example (a, b);

    inout a, b;
    tran t1 (a, b);

endmodule
```

The `tran` primitive continuously connects the two nets. A change on either side can propagate to the other side according to Verilog net and strength-resolution rules.

7.2 The tranif1 Primitive

The `tranif1` primitive is a control-enabled bidirectional switch.

```
tranif1 instance_name (net1, net2, control);
```

The switch is enabled when the control is 1.

```
module tranif1_example (a, b, en);

    inout a, b;
    input en;

    tranif1 t1 (a, b, en);

endmodule
```

When `en = 1`, the two nets are connected. When `en = 0`, they are disconnected.

7.3 The tranif0 Primitive

The `tranif0` primitive is enabled when its control input is 0.

```
module tranif0_example (a, b, en);

    inout a, b;
    input en;

    tranif0 t1 (a, b, en);

endmodule
```

The control behavior is summarized below.

Primitive	Control = 0	Control = 1
`tran`	Always ON	Always ON
`tranif1`	OFF	ON
`tranif0`	ON	OFF

8 Resistive Switches

Verilog provides resistive switch primitives to model weaker signal propagation than that of their non-resistive counterparts.

Common resistive switch primitives include:

- `rnmos`
- `rpmos`
- `rtran`
- `rtranif0`
- `rtranif1`

8.1 Example: Resistive NMOS

```
module rnmos_example (y, a, en);

    input a, en;
    output y;

    rnmos r1 (y, a, en);

endmodule
```

The `rnmos` primitive is the resistive counterpart of `nmos`. It uses the Verilog strength rules for resistive switches.

8.2 Example: Resistive Bidirectional Switch

```
module rtran_example (a, b);

    inout a, b;

    rtran r1 (a, b);

endmodule
```

The `rtran` primitive provides a bidirectional resistive connection.

Resistive primitives model discrete drive strength effects. They do not perform analog resistance or current calculations.

9 Pull-Up and Pull-Down Primitives

Pull devices provide weak logic values on nets.

9.1 Pull-Up Primitive

The `pullup` primitive supplies a logic one with pull strength.

```
module pullup_example (y);

    output y;
    wire y;

    pullup p1 (y);

endmodule
```

A pull-up is useful when a net should default to logic one in the absence of a stronger driver.

9.2 Pull-Down Primitive

The `pulldown` primitive supplies a logic zero with pull strength.

```
module pulldown_example (y);

    output y;
    wire y;

    pulldown p1 (y);

endmodule
```

A pull-down is useful when a net should default to logic zero.

9.3 Pull-Up and Pull-Down Comparison

Primitive	Default Logic Value
`pullup`	1
`pulldown`	0

If multiple drivers are connected to the same net, the final value depends on their drive strengths and the Verilog net-resolution rules.

10 Example: Switch-Level AND Gate

An AND function can be constructed using an NMOS pull-down network and a pull-up device. However, a simple transistor network must be designed carefully to ensure that the output has the intended behavior for every input combination.

The following example uses a CMOS-style switch network for a two-input NAND gate.

10.1 NAND Gate Using Switches

```
module switch_nand (y, a, b);

    input a, b;
    output y;

    supply1 vdd;
    supply0 gnd;
    wire n1;

    pmos p1 (y, vdd, a);
    pmos p2 (y, vdd, b);

    nmos n1m (y, n1, a);
    nmos n2m (n1, gnd, b);

endmodule
```

The two PMOS devices form a parallel pull-up network, while the two NMOS devices form a series pull-down network.

The intended NAND truth table is:

a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

In switch-level simulation, internal net declarations and connections should be checked carefully. The transistor network must not introduce unintended multiple drivers or incorrect node connections.

11 Example: Switch-Level Multiplexer

A transmission-gate multiplexer can select one of two input signals using complementary select signals.

11.1 Verilog Code

```
module switch_mux2 (y, a, b, sel, sel_bar);

    input a, b, sel, sel_bar;
    output y;
    wire n1, n2;
    tranif0 t1 (n1, a, sel);
    tranif1 t2 (n2, b, sel);
    tranif1 t3 (y, n1, sel_bar);
    tranif0 t4 (y, n2, sel_bar);
endmodule
```

This example illustrates conditional signal connections. For a practical multiplexer, use a carefully verified switch network with consistent complementary controls and defined output behavior.

A behavioral implementation is generally easier to verify:

```
module mux2 (y, a, b, sel);

    input a, b, sel;
    output y;

    assign y = sel ? b : a;

endmodule
```

12 Testbench for Switch-Level Modeling

A testbench can apply input values and observe the outputs of a switch-level circuit.

12.1 Example: NMOS Testbench

```
module nmos_test;

    reg a, en;
    wire y;

    nmos_switch dut (y, a, en);

    initial begin
        a = 0; en = 0;
        #10 en = 1;
        #10 a = 1;
        #10 a = 0;
        #10 en = 0;
        #10 $finish;
    end

    initial begin
        $monitor("time=%0t a=%b en=%b y=%b",
                 $time, a, en, y);
    end

endmodule
```

The testbench changes the input and enable signals and monitors the output.

When the NMOS is off, the output may become high impedance if no other driver is connected. When it is on, the output follows the input subject to the Verilog switch and strength semantics.

13 Logic Strengths and High-Impedance State

Switch-level modeling supports the four-state logic values:

- `0`: Logic zero
- `1`: Logic one
- `x`: Unknown logic value
- `z`: High-impedance state

High impedance is particularly important for tri-state buses and disconnected switch terminals.

Verilog also defines drive strengths such as `strong`, `pull`, and `weak`. These strengths influence how multiple drivers are resolved on a net.

For example, a pull-up may establish a default logic one, while a stronger active driver can override it.

The simulator resolves the resulting net value according to the applicable drive strengths, switch behavior, and net-resolution rules.

14 Switch-Level Primitive Summary

Primitive	Type	Behavior
`nmos`	MOS switch	Conducts when control is 1.
`pmos`	MOS switch	Conducts when control is 0.
`cmos`	CMOS switch	Combines NMOS and PMOS switching paths.
`rnmos`	Resistive MOS switch	Resistive counterpart of NMOS.
`rpmos`	Resistive MOS switch	Resistive counterpart of PMOS.
`tran`	Bidirectional switch	Connects two nets continuously.
`tranif1`	Controlled switch	Connects two nets when control is 1.
`tranif0`	Controlled switch	Connects two nets when control is 0.
`rtran`	Resistive switch	Bidirectional resistive connection.
`rtranif1`	Resistive controlled switch	Resistive connection enabled by control 1.
`rtranif0`	Resistive controlled switch	Resistive connection enabled by control 0.
`pullup`	Pull device	Provides a pull-strength logic one.
`pulldown`	Pull device	Provides a pull-strength logic zero.

15 Switch-Level Modeling vs. Other Modeling

Feature	Switch-Level	Behavioral / RTL
Description	Uses switch and transistor primitives.	Describes logic behavior and operations.
Abstraction	Lower-level digital abstraction.	Higher-level functional abstraction.
Typical constructs	`nmos`, `pmos`, `tran`.	`assign`, `always`, `case`.
Signal resolution	Can model switch connectivity and drive strengths.	Usually relies on ordinary logic expressions and assignments.
Applications	Pass gates, transistor networks, tri-state connections.	RTL design, controllers, datapaths.

16 Limitations of Switch-Level Modeling

- Switch primitives model digital logic and drive strengths, not full transistor electrical behavior.
- They do not directly calculate analog voltage, current, or transistor capacitance.
- Complex switch networks can be difficult to debug and maintain.
- Unknown and high-impedance values can propagate through connected networks.
- Multiple drivers can produce unexpected results if net strengths are not considered.
- Some synthesis tools do not support switch primitives for ordinary RTL synthesis.
- Simulation support can vary by tool and language mode.

17 RTL Design Guidelines

- Use `nmos` and `pmos` for simple transistor-level switching models.
- Use `cmos` to represent a combined CMOS switch structure.
- Use `tranif0` and `tranif1` for controlled bidirectional connections.
- Use `pullup` and `pulldown` when modeling weak default logic levels.
- Check all net declarations and terminal connections carefully.
- Consider unknown values, high impedance, and drive-strength resolution during simulation.
- Verify that complementary control signals are consistent when implementing transmission gates.
- Prefer behavioral or RTL descriptions for portable, synthesizable digital designs.
- Check simulator and synthesis-tool support before using switch primitives in a project.

18 Summary

- Switch-level modeling describes digital circuits using built-in switch primitives.
- `nmos` and `pmos` model controlled MOS switching paths.
- `cmos` combines complementary NMOS and PMOS switching behavior.
- `tran`, `tranif0`, and `tranif1` model bidirectional connections.
- Resistive switches model weaker signal propagation according to Verilog strength rules.
- `pullup` and `pulldown` establish default logic levels.
- Switch-level simulation is useful for studying connectivity, pass gates, and net resolution.

Key takeaway: Switch-level modeling is useful when the connectivity and switching behavior of transistor-like devices matter. For most modern RTL design tasks, behavioral or structural modules are easier to maintain, verify, and synthesize.